Class Project Phase II Submission Template

1. Document explaining and justifying design patterns you have identified for this project. A minimum of three design patterns are required.

Design Pattern 1: Factory Method

I chose the Factory Method as one of the patterns for the SmartCity Hub to simplify and centralize the creation of different types of users. SmartCity Hub supports multiple user types such as Citizen, CityAdministrator, and DepartmentManager. Although these users have different responsibilities, they share common characteristics that are represented by the User superclass. So instead of requiring the SmartCityController or user interface to directly create each specific type of user, the system hands this responsibility to the UserFactory.

UserFactory provides the createUser(type : String) : User method. Based on the requested user type, the factory can create and return the appropriate subclass of User. The factory can create a Citizen, CityAdministrator, or DepartmentManager depending on the type of account being registered. The Controller uses its registerUser() operation to communicate with the factory rather than directly constructing the different user objects.

Using the Factory Method pattern improves the design by separating object creation from the classes that use those objects. Without the factory, user-creation logic would become distributed and require conditionals wherever a user needs to be created. Using a Factory Pattern here reduces coupling and makes the application easier to maintain, while also improving extensibility. If my Hub later introduces another user type, such as a city worker or city official, the new user class can be incorporated into the factory without requiring major changes to the application. So, the Factory Method pattern provides a much cleaner and more maintainable approach compared to creating the different user types required by SmartCity Hub.

Design Pattern 2: Observer

I chose the Observer design pattern to be used in my SmartCity Hub to support notifications when important information changes. Citizens may need to be informed when the status of a submitted service request changes or when the city publishes a new announcement or even a city-wide state of emergency. Rather than coupling the Citizen class directly to ServiceRequest and Announcement notification logic, I used the Observer pattern to distribute these updates.

In my refined class diagram, the Observer interface defines an update, which is a String message, operation. The Citizen class implements this interface, allowing a citizen to receive notifications. The Subject interface defines operations for attaching observers, detaching observers, and also notifying registered observers. Then NotificationManager implements the Subject interface and maintains a collection of observers. When an

Announcement is published or the status of a ServiceRequest changes, the NotificationManager can notify the registered observers.

This pattern improves the SmartCity Hub design because the objects producing changes do not need to know about every object interested in those changes. This results in a lower coupling between notification sources and the recipients. It also makes the notification system easier to extend a well. In future versions the SmartCity Hub could add email notifications, mobile notifications, or other types of subscribers without having to redesign the service-request or announcement classes.

The Observer pattern works particularly well for SmartCity Hub because city information changes over time and users benefit from receiving updates without repeatedly checking the system. Separating notifications into Observer, Subject, and Notification Manager also prevents notification logic from becoming scattered and provides a more organized and maintainable design.

Design Pattern 3: Strategy

The Strategy design pattern I used in SmartCity Hub provides different methods for filtering community events. The project allows citizens to browse community events and apply filters when searching for events. Using this design, I added two filtering approaches, where citizens can filter by date and event type. Instead of placing all possible filtering configurations directly inside SmartCityController, the Strategy pattern separates each algorithm into its own class.

The EventFilterStrategy interface defines the common filter which returns a list. Then the DateFilterStrategy and EventTypeFilterStrategy implement this interface and provide different implementations that match the filter type. The SmartCityController works with the EventFilterStrategy interface rather than depending directly on one specific filtering algorithm. As a result, the controller can use the appropriate strategy depending on the filter selected by the citizen.

The primary benefit of this pattern is flexibility and extensibility, which you almost always want. Without the Strategy pattern, the controller could eventually contain a large number of conditional statements for determining whether events should be filtered by date, event type, location, cost, or other criteria. As additional filtering options are introduced, that method would become increasingly difficult to maintain and test.

With the Strategy pattern, new filtering methods can instead be implemented as additional strategy classes without modifying the existing controller much. For example, LocationFilterStrategy or PopularityFilterStrategy could later implement the same EventFilterStrategy interface. Each filtering algorithm can also be tested independently. This keeps the controller focused on coordinating application behavior while individual strategy classes are responsible for performing specific filtering operations. Therefore, Strategy provides SmartCity Hub with a cleaner and more adaptable approach to event searching.

2. Refined class diagram clearly shows the design pattern related to classes and other classes. Astah must be used to develop the class diagram. (Copy and paste your class diagram here. Make sure to use the correct UML notations. Points will be given for the completeness and the correctness of the diagram)

